

딥러닝 파이토치 교과서 6장 Part 1 (~6.1.4)

6장 합성곱 신경망 II

6.1 이미지 분류를 위한 신경망

입력 데이터로 이미지를 사용한 분류(classification)는 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것이다. 이미지 분류(image classification)에서 주로 사용되는 합성곱 신경망의 유형을 알아본다.

6.1.1 LeNet-5

LeNet-5는 합성곱 신경망이라는 개념을 최초로 안 르쿤(Yann LeCun)이 개발한 구조다. 1995년 안 르쿤, 레옹 보토(Leon Bottu), 요슈아 벤지오(Yosua Bengio), 패트릭 하프너(Patrick Haffner)가 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5를 발표했는데, 그것이 현재 CNN의 초석이 되었다.

LeNet-5는 합성곱(convolutional)과 다운 샘플링(sub-sampling)(혹은 풀링)을 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행한다.

- C로 시작하는 것은 합성곱층을 의미한다.
- S로 시작하는 것은 풀링층을 의미한다.
- F로 시작하는 것은 완전연결층을 의미한다.

구체적으로 살펴보면 C1에서 5×5 합성곱 연산 후 28×28 크기의 특성 맵 여섯 개를 생성한다. S2에서 다운 샘플링하여 특성 맵 크기를 14×14 로 줄인다. 다시 C3에서 5×5 합성곱 연산하여 10×10 크기의 특성 맵 16개를 생성하고, S4에서 다운 샘플링하여 특성 맵 크기를 5×5 로 줄인다. C5에서 5×5 합성곱 연산하여 1×1 크기의 특성 맵 120개를 생성하고, 마지막으로 F6에서 완전연결층으로 C5의 결과를 유닛(또는 노드) 84개에 연결시킨다.

32×32 크기의 이미지에 합성곱층과 최대 풀링층 쌍으로 두 번 적용된 후 완전연결층을 거쳐 이미지가 분류되는 신경망이다.

LeNet-5 예제 신경망의 계층 상세는 다음과 같다.

1. 이미지: 32×32
2. 합성곱층: 특성 맵 6개, 크기 28×28 , 커널 크기 5×5 , 스트라이드 1, 활성화 함수 렐루(ReLU)
3. 최대 풀링층: 특성 맵 6개, 크기 14×14 , 커널 크기 2×2 , 스트라이드 2
4. 합성곱층: 특성 맵 16개, 크기 10×10 , 커널 크기 5×5 , 스트라이드 1, 활성화 함수 렐루(ReLU)
5. 최대 풀링층: 특성 맵 16개, 크기 5×5 , 커널 크기 2×2 , 스트라이드 2
6. 완전연결층: 120, 활성화 함수 렐루(ReLU)
7. 완전연결층: 84, 활성화 함수 렐루(ReLU)
8. 완전연결층: 2, 활성화 함수 소프트맥스(softmax)

'tqdm'은 아랍어로 progress(진행 상태)라고도 한다. 즉, 진행 상태를 바(bar) 형태로 가시화하여 보여 준다. 주로 모델 훈련에 대한 진행 상태를 확인하고자 할 때 사용한다.

torchsummary는 케라스와 같은 형태로 모델을 출력해 볼 수 있는 라이브러리다. summary를 통해 모델의 네트워크 관련 정보(학습 가능한 파라미터 수, 입력 크기, 네트워크의 총 크기 등)를 확인할 수 있다.

Conv2d 계층에서의 출력 크기 구하는 공식과 MaxPool2d 계층에서의 출력 크기 구하는 공식은 5장에서 다룬 것과 동일하다.

- Conv2d 출력 크기 = $(W - F + 2P) / S + 1$
- MaxPool2d 출력 크기 = IF / F

모델을 학습시킬 때 경사 하강법으로 모멘텀 SGD를 사용할 수 있다. 모멘텀 SGD는 SGD에 관성이 추가된 것으로 매번 기울기를 구하지만 가중치를 수정하기 전에 이전 수정 방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정되게 하는 방법이다. 이때 사용되는 파라미터는 다음과 같다.

1. 첫 번째 파라미터: 경사 하강법을 통해 궁극적으로 업데이트하고자 하는 파라미터(가중치와 바이어스)
2. lr(learning rate): 가중치를 변경할 때 얼마나 크게 변경할지 결정
3. momentum: SGD를 적절한 방향으로 가속화하며 흔들림(진동)을 줄여 주는 매개변수

손실 함수의 reduction 파라미터 기본값은 'mean'이다. 'mean'은 정답과 예측 값의 오차를 구한 후 그 값들의 평균을 반환한다. 즉, 손실 함수 특성상 전체 오차를 배치 크기로 나눔으로써 평균을 반환하기 때문에 epoch_loss를 계산하는 동안 loss.item()과 inputs.size(0)을 곱해 준다.

model.train()과 model.eval()은 각각 모델을 학습 모드와 평가 모드로 전환한다. 모델 평가(테스트) 시에는 with torch.no_grad()를 사용하는데, 이는 역전파 과정 중 텐서들에 대한 변화도를 계산할 필요가 없음을 나타내는 것으로, 훈련 데이터셋의 모델 학습과 가장 큰 차이점이다.

소프트맥스(softmax)는 지정된 차원(dim)을 따라 텐서의 요소(텐서의 개별 값)가 (0, 1) 범위에 있고 합계가 1이 되도록 크기를 다시 조정한다.

6.1.2 AlexNet

AlexNet은 ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회인 'ILSVRC 2012'에서 우승한 CNN 구조다.

CNN은 3차원 구조를 갖는다는 것을 이해해야 한다(이미지를 다루기 때문에 기본적으로 3차원 데이터를 다룬다). 이미지 크기를 나타내는 너비(width)와 높이(height)뿐만 아니라 깊이(depth)를 갖는다. 보통 색상이 많은 이미지는 R/G/B 성분 세 개를 갖기 때문에 시작이 3이지만, 합성곱을 거치면서 특성 맵이 만들어지고 이것에 따라 중간 영상의 깊이가 달라진다.

AlexNet은 합성곱층 총 다섯 개와 완전연결층 세 개로 구성되어 있으며, 맨 마지막 완전연결층은 카테고리 1000 개를 분류하기 위해 소프트맥스 활성화 함수를 사용하고 있다. 전체적으로 보면 GPU 두 개를 기반으로 한 병렬 구조인 점을 제외하면 LeNet-5와 크게 다르지 않다.

AlexNet 구조 상세는 다음과 같다.

1. 이미지: 227×227
2. 합성곱층: 특성 맵 96개, 크기 55×55 , 커널 크기 11×11 , 스트라이드 4, 활성화 함수 렐루(ReLU)
3. 최대 풀링층: 특성 맵 96개, 크기 27×27 , 커널 크기 3×3 , 스트라이드 2
4. 합성곱층: 특성 맵 256개, 크기 27×27 , 커널 크기 5×5 , 스트라이드 1, 활성화 함수 렐루(ReLU)
5. 최대 풀링층: 특성 맵 256개, 크기 13×13 , 커널 크기 3×3 , 스트라이드 2

6. 합성곱층: 특성 맵 384개, 크기 13×13 , 커널 크기 3×3 , 스트라이드 1, 활성화 함수 렐루(ReLU)
7. 합성곱층: 특성 맵 384개, 크기 13×13 , 커널 크기 3×3 , 스트라이드 1, 활성화 함수 렐루(ReLU)
8. 합성곱층: 특성 맵 256개, 크기 13×13 , 커널 크기 3×3 , 스트라이드 1, 활성화 함수 렐루(ReLU)
9. 최대 풀링층: 특성 맵 256개, 크기 6×6 , 커널 크기 3×3 , 스트라이드 2
10. 완전연결층: 4096, 활성화 함수 렐루(ReLU)
11. 완전연결층: 4096, 활성화 함수 렐루(ReLU)
12. 완전연결층: 1000, 활성화 함수 소프트맥스(softmax)

네트워크에는 학습 가능한 변수가 총 6600만 개 있다. 네트워크에 대한 입력은 $227 \times 227 \times 3$ 크기의 RGB 이미지이며, 각 클래스(혹은 카테고리)에 해당하는 1000×1 확률 벡터를 출력한다.

AlexNet의 첫 번째 합성곱층 커널의 크기는 $11 \times 11 \times 3$ 이며, 스트라이드를 4로 적용하여 특성 맵을 96개 생성하기 때문에 $55 \times 55 \times 96$ 의 출력을 갖는다. 첫 번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관없는 정보를 추출하기 위한 커널이 학습되고, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습된다.

AlexNet은 파라미터를 6000만 개 사용하는 모델이다. 이때 충분한 데이터가 없으면 과적합이 발생하는 등 테스트 데이터에 대한 성능이 좋지 않을 수 있다.

`nn.AdaptiveAvgPool2d`는 `nn.AvgPool2d`처럼 풀링을 위해 사용한다. `AvgPool2d`에서는 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작한다. 반면에 `AdaptiveAvgPool2d`는 풀링 작업이 끝날 때 필요한 출력 크기를 정의한다. 즉, `AvgPool2d`에서는 커널 크기, 스트라이드, 패딩을 지정했다면 `AdaptiveAvgPool2d`는 출력에 대한 크기만 지정한다. 출력 크기가 정해졌기 때문에 커널 크기, 스트라이드, 패딩 값을 구할 수 있다. 따라서 `AdaptiveAvgPool2d`를 사용할 경우 출력 크기에 대한 조정이 상당히 쉬워진다. 참고로 `AdaptiveAvgPool2d`는 입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우 유용하다.

ReLU 활성화 함수에서 inplace 연산이란 연산에 대한 결괏값을 새로운 변수에 저장하는 것이 아닌 기존 데이터를 대체하는 것을 의미한다. 즉, 기존 값을 연산 결괏값으로 대체함으로써 기존 값들을 무시하겠다는 의미다.

6.1.3 VGGNet

VGGNet은 카렌 시모니안(Karen Simonyan)과 앤드류 지서만(Andrew Zisserman)이 2015 ICLR에 게재한 "Very deep convolutional networks for large-scale image recognition" 논문에서 처음 발표했다.

VGGNet은 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하려고 탄생했다. 즉, 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 것이 VGG다. VGG 연구 팀은 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3×3 으로 고정했다.

네트워크 계층의 총 개수에 따라 여러 유형의 VGGNet(VGG16, VGG19 등)이 있다.

VGG16에는 파라미터가 총 1억 3300만 개 있다. 모든 합성곱 커널의 크기는 3×3 , 최대 풀링 커널의 크기는 2×2 이며, 스트라이드는 2다. 결과적으로 64개의 224×224 특성 맵($224 \times 224 \times 64$)들이 생성된다. 또한, 마지막 16 번째 계층을 제외하고는 모두 ReLU 활성화 함수가 적용된다.

VGG16 구조 상세는 다음과 같다.

1. 이미지: 224×224
2. 합성곱층 $\times 2$: 특성 맵 64개, 크기 224×224 , 커널 크기 3×3 , 스트라이드 1, 활성화 함수 렐루(ReLU)
3. 최대 풀링층: 특성 맵 64개, 크기 112×112 , 커널 크기 2×2 , 스트라이드 2

4. 합성곱층 ×2: 특성 맵 128개, 크기 112×112, 커널 크기 3×3, 스트라이드 1, 활성화 함수 렐루(ReLU)
5. 최대 풀링층: 특성 맵 128개, 크기 56×56, 커널 크기 2×2, 스트라이드 2
6. 합성곱층 ×4: 특성 맵 256개, 크기 56×56, 커널 크기 3×3, 스트라이드 1, 활성화 함수 렐루(ReLU)
7. 최대 풀링층: 특성 맵 256개, 크기 28×28, 커널 크기 2×2, 스트라이드 2
8. 합성곱층 ×4: 특성 맵 512개, 크기 28×28, 커널 크기 3×3, 스트라이드 1, 활성화 함수 렐루(ReLU)
9. 최대 풀링층: 특성 맵 512개, 크기 14×14, 커널 크기 2×2, 스트라이드 2
10. 합성곱층 ×4: 특성 맵 512개, 크기 14×14, 커널 크기 3×3, 스트라이드 1, 활성화 함수 렐루(ReLU)
11. 최대 풀링층: 특성 맵 512개, 크기 7×7, 커널 크기 2×2, 스트라이드 2
12. 완전연결층: 4096, 활성화 함수 렐루(ReLU)
13. 완전연결층: 4096, 활성화 함수 렐루(ReLU)
14. 완전연결층: 1000, 활성화 함수 소프트맥스(softmax)

VGGNet 중에서 가장 간단한 VGG11을 파이토치로 구현할 수 있다. VGG16이나 VGG19를 구현하려면 VGG11에서 더 깊게 쌓아 올리면 된다.

VGG11, VGG13, VGG16, VGG19 모델의 계층을 정리할 때 숫자(output channel, 출력 채널)는 Conv2d를 수행하라는 의미이며, 출력 채널(output channel)이 다음 계층의 입력 채널(input channel)이 된다. 또한, M은 최대 풀링(max pooling)을 수행하라는 의미다.

- VGG11: 8(합성곱층) + 3(풀링층) = 11(전체 계층)
- VGG13: 10(합성곱층) + 3(풀링층) = 13(전체 계층)
- VGG16: 13(합성곱층) + 3(풀링층) = 16(전체 계층)
- VGG19: 16(합성곱층) + 3(풀링층) = 19(전체 계층)

batch_norm(Batch Normalization)은 데이터의 평균을 0으로, 표준편차를 1로 분포시키는 것이다. 각 계층에서 입력 데이터의 분포는 앞 계층에서 업데이트된 가중치에 따라 변한다. 즉, 각 계층마다 변화되는 분포는 학습 속도를 늦출 뿐만 아니라 학습도 어렵게 한다. 따라서 각 계층의 입력에 대한 분산을 평균 0, 표준편차 1로 분포시키는 것이 batch_norm(배치 정규화)이다.

VGG 모델은 사전 훈련된 모델이다. 이미 누군가가 대용량의 이미지 데이터로 학습을 시켰으며, 최상의 상태로 튜닝을 거쳐 모든 사람이 사용할 수 있도록 공유한 사전 훈련된 모델이다. 따라서 코드 한 줄로 쉽게 가져다 사용할 수 있다. vgg11_bn은 VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용하겠다는 의미이며, pretrained를 True로 설정하면 사전 훈련된 모델을 사용(미리 학습된 파라미터 값들을 사용)하겠다는 의미다.

ImageFolder는 계층적인 폴더 구조를 가지고 있는 데이터셋을 불러올 때 사용한다.

6.1.4 GoogLeNet

GoogLeNet은 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망이다.

깊고 넓은 신경망을 위해 GoogLeNet은 인셉션(inception) 모듈을 추가했다. 인셉션 모듈에서는 특징을 효율적으로 추출하기 위해 1×1, 3×3, 5×5의 합성곱 연산을 각각 수행한다. 3×3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩을 추가해야 한다. 결과적으로 GoogLeNet에 적용된 해결 방법은 희소 연결(sparse connectivity)이다.

CNN은 합성곱, 풀링, 완전연결층들이 서로 밀집(dense)(정교하고 뽁뽁하게)하게 연결되어 있다. 뽁뽁하게 연결된 신경망 대신 관련성(correlation)이 높은 노드끼리만 연결하는 방법을 희소 연결이라고 한다. 이것으로 연산량이 적어지며 과적합도 해결할 수 있다.

인셉션 모듈의 네 가지 연산은 다음과 같다.

1. 1×1 합성곱
2. 1×1 합성곱 + 3×3 합성곱
3. 1×1 합성곱 + 5×5 합성곱
4. 3×3 최대 풀링(maxpooling) + 1×1 합성곱(convolutional)

딥러닝을 이용하여 ImageNet과 같은 대회에 참여하거나 서비스를 제공하려면 대용량 데이터를 학습해야 한다. 심층 신경망의 아키텍처에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면) 인식률은 좋아지지만, 과적합이나 기울기 소멸 문제(vanishing gradient problem)를 비롯한 학습 시간 지연과 연산 속도 등의 문제가 있다. 특히 합성곱 신경망에서 이러한 문제들이 자주 나타나는데, GoogLeNet(혹은 인셉션이라고도 불림)으로 이러한 문제를 해결할 수 있다고 생각하면 된다.